

BAOCAO-luong-tu-hoa-nhung

HQC – LƯỢNG TỬ HÓA & TRIỂN KHAI NHÚNG (bước 4-5-6 khung Embedded AI)

Trả lời trực tiếp: mô hình có lượng tử hóa được không? nhúng được không? CÓ CẢ HAI – và đã kiểm chứng bằng mã C biên dịch chạy thật, không phải ước lượng.

1. Mô hình hiện tại là gì (về mặt nhúng)

PCA k=6 trên 24 đặc trưng. Toàn bộ mô hình là **3 mảng số**:

Thành phần	Kích thước	Vai trò
W (thành phần chính)	$6 \times 24 = 144$ số	phép chiếu
mean_	24 số	tâm không gian PCA
mu , sd	24 + 24 số	chuẩn hóa z-score
TỔNG	216 tham số = 864 byte (float32)	

Một lần suy luận: 144 nhân-cộng (chiếu) + 144 nhân-cộng (tái tạo) + 96 phép cộng/trừ/abs = **~384 phép toán số học**. **Không có hàm siêu việt** (không exp, tanh, sqrt, không softmax).

→ Đây là lý do mô hình này **cực kỳ thân thiện với nhúng** – nó chỉ là hai phép nhân ma trận nhỏ. So sánh: một autoencoder 1.164 trọng số cần cả hàm `tanh` (bảng tra hoặc xấp xỉ đa thức trên MCU); ED-LSTM của paper trụ cần `sigmoid` + `tanh` × nhiều cổng × nhiều bước thời gian.

2. Lượng tử hóa: một thất bại có giá trị, rồi thành công

Thử 1 – Int8 per-tensor cho TẤT CẢ: SẬP HOÀN TOÀN

Lượng tử hóa mù per-tensor (một hệ số scale cho cả mảng) làm mô hình trả về NaN . Nguyên nhân đã truy được:

Mảng	Dải giá trị	Tỉ số max/min
mu	-0,92 ... 47,57	343×
sd	0,0051 ... 2,01	396×
W	-0,495 ... 0,651	180×

Với sd : scale = $2,01/127 = 0,0159$, nên mọi phần tử $sd < 0,0079$ bị làm tròn về 0 → **3/24 phần tử về 0** → chia cho 0 → NaN . (Các sd nhỏ nhất: $0,0051 \cdot 0,0053 \cdot 0,0066$.)

Đây là bài học phương pháp đáng viết vào báo cáo: lượng tử hóa không phải thao tác cơ học áp lên mọi tham số. Tham số ở **mẫu số** (như sd) và tham số có **dải động rộng** phải xử lý riêng.

Thử 2 & 3 – Phương án đúng

Phương án	Bộ nhớ	AUC	Sai lệch residual so float32
float32 (gốc)	864 B	0,8337	—
int8 per-tensor tất cả	216 B	SẬP	—
int8 W per-channel, mu / sd float32	360 B	0,8338	TB 0,0028 · max 0,044
int8 W per-channel + mu / sd float16	264 B	0,8339	TB 0,0032 · max 0,039

Nén 3,3× (864 → 264 byte), **AUC KHÔNG mất gì** (0,8337 → 0,8339 — thay đổi trong sai số ngẫu nhiên).

Cách làm: W lượng tử hóa **per-channel** (mỗi hàng một scale, 6 scale thay 1); mu / sd giữ **float16** (48 số = 96 byte, không đáng đổi lấy rủi ro chia-0).

3. Nhúng: đã sinh mã C và KIỂM CHỨNG

Sinh file hqc_detector.c — C thuần, không thư viện ngoài, không malloc :

Flash (hằng số) : 384 byte ($W_q \cdot 144 + W_{scale} \cdot 24 + mean_q \cdot 24 + \mu/sd \cdot 192$)
RAM (stack) : 216 byte ($z[24] + t[6] + r[24] \cdot float$)
Mã máy hàm : 1.149 byte (gcc -O2, x86-64)

Kiểm chứng đúng đắn — không chỉ biên dịch được mà chạy ĐÚNG

Biên dịch `gcc -O2 -std=c99` và so 6 mẫu với Python:

Mẫu	C	Python	Sai lệch
0	0,668439	0,668437	1,9e-06
1	7,013474	7,013483	9,5e-06
2	0,499448	0,499447	1,6e-06
3	3,909388	3,909396	8,6e-06
4	5,836287	5,836296	9,1e-06
5	3,478087	3,478093	6,0e-06

Tất cả khớp, sai lệch $< 1e-5$ (do thứ tự cộng dồn float, không phải lỗi thuật toán).

Vừa với phần cứng nào

Vi điều khiển	Flash	RAM	Vừa?
ATmega328 (Arduino Uno)	32 KB	2 KB	✓ dùng 1,2% Flash, 11% RAM
STM32F103 (Cortex-M3)	64 KB	20 KB	✓ dư thừa
ESP32	4 MB	520 KB	✓ dư rất nhiều
Ngay cả ATtiny1614	16 KB	2 KB	✓ vừa

Thời gian: đo trên x86 là 30 μs /mẫu (numpy có overhead). Trên Cortex-M4 @80 MHz, 384 phép toán float $\approx$ **15–50 μs** . Cadence dữ liệu là **1 giờ** → dư thời gian gấp **hàng chục triệu lần**. → Có thể chạy trên MCU chậm nhất, hoặc để MCU ngủ 99,999% thời gian (quan trọng nếu chạy pin).

4. Trả lời gọn ba câu hỏi

Câu hỏi	Trả lời
---------	---------

Lượng tử hóa được không?	Được — Int8 per-channel, nén 3,3×, không mất độ chính xác. Nhưng KHÔNG được lượng tử hóa mù per-tensor (sẽ sập).
Nhúng được không?	Được — 384 B Flash + 216 B RAM + 1,1 KB mã. Vừa cả ATmega328. Đã biên dịch và kiểm chứng khớp Python.
Có phù hợp môn học không?	Rất phù hợp — làm được trọn bước 4 (nén), 5 (C-array lên MCU), 6 (đo dấu chân + thời gian). Và có một thất bại lượng tử hóa thật để phân tích, giá trị hơn một bài "quantize xong, chạy tốt".

5. Điểm mạnh so với paper trụ

Paper trụ (Entropy 2021) xếp **ED-LSTM** và **VAE** là tốt nhất — nhưng **không làm phần nhúng**. Hai kiến trúc đó rất khó nhúng: hàng nghìn trọng số, cần `sigmoid` / `tanh`, LSTM còn cần giữ trạng thái qua nhiều bước thời gian.

Mô hình của ta **cố tình nhỏ** (chọn bằng learning curve, không phải chọn cho dễ nhúng) — và hóa ra chính vì nhỏ mà nhúng được. Đây là một luận điểm đứng vững: **với 743 giờ dữ liệu nèn, mô hình đúng kích thước vừa cho kết quả tốt hơn vừa triển khai được lên vi điều khiển 2 KB RAM.**

6. Việc còn lại của nhánh nhúng

- ☐ Biên dịch chéo `arm-none-eabi-gcc` để có số Flash/RAM thật của ARM (hiện là số x86)
- ☐ Đo điện năng thực tế trên board (bước 6 của khung môn học)
- ☐ Thêm tầng ngưỡng kiểm soát (SPE limit) vào mã C để nó ra **quyết định** báo động, không chỉ ra số residual
- ☐ Cân nhắc số nguyên hoàn toàn (fixed-point Q15) nếu MCU không có FPU — hiện mã dùng float